

```

"""
财报PDF下载器
支持：巨潮资讯直接下载 / Tushare披露接口
"""

import time
import logging
import requests
import tushare as ts
from pathlib import Path
from typing import Optional

logger = logging.getLogger(__name__)

class ReportDownloader:
    """
    A股年报PDF下载器

    使用方法：
        downloader = ReportDownloader(tushare_token='xxx', save_dir='./reports')
        downloader.download('600893.SH', 2024)
        downloader.batch_download(['600893.SH', '601021.SH'], [2023, 2024])
    """

    # 巨潮资讯年报搜索接口
    CNINFO_SEARCH_URL = "http://www.cninfo.com.cn/new/hisAnnouncement/query"
    CNINFO_FILE_URL = "http://static.cninfo.com.cn/"

    HEADERS = {
        "User-Agent": (
            "Mozilla/5.0 (X11; Linux x86_64) "
            "AppleWebKit/537.36 (KHTML, like Gecko) "
            "Chrome/120.0.0.0 Safari/537.36"
        ),
        "Referer": "http://www.cninfo.com.cn/",
    }

    def __init__(self, tushare_token: str, save_dir: str = "./reports"):
        self.save_dir = Path(save_dir)
        self.save_dir.mkdir(parents=True, exist_ok=True)
        self.pro = ts.pro_api(tushare_token)

    # -----

```

```

# 公开接口
# -----

def download(self, ts_code: str, year: int) -> Optional[Path]:
    """下载单份年报，返回本地PDF路径；已存在则跳过"""
    save_path = self._pdf_path(ts_code, year)
    if save_path.exists():
        logger.info(f"[skip] {ts_code} {year}年报已存在: {save_path}")
        return save_path

    logger.info(f"[download] {ts_code} {year}年报 ...")

    # 优先用巨潮直接抓，失败再fallback到Tushare
    path = self._download_from_cninfo(ts_code, year, save_path)
    if path is None:
        path = self._download_from_tushare(ts_code, year, save_path)

    if path:
        logger.info(f"[ok] 保存至 {path}")
    else:
        logger.warning(f"[fail] {ts_code} {year}年报下载失败")
    return path

def batch_download(
    self,
    stock_list: list[str],
    years: list[int],
    sleep_sec: float = 3.0,
) -> dict[str, Path]:
    """批量下载，返回 {ts_code_year: path} 字典"""
    results = {}
    for ts_code in stock_list:
        for year in years:
            key = f"{ts_code}_{year}"
            path = self.download(ts_code, year)
            if path:
                results[key] = path
            time.sleep(sleep_sec)
    return results

# -----
# 内部方法
# -----

def _pdf_path(self, ts_code: str, year: int) -> Path:
    d = self.save_dir / ts_code
    d.mkdir(parents=True, exist_ok=True)

```

```

        return d / f"{year}_annual_report.pdf"

def _download_from_cninfo(
    self, ts_code: str, year: int, save_path: Path
) -> Optional[Path]:
    """从巨潮资讯抓取年报PDF"""
    # 将 600893.SH → 600893
    code = ts_code.split(".")[0]
    exchange = "sh" if ts_code.endswith("SH") else "sz"
    org_id = self._get_cninfo_org_id(code, exchange)
    if not org_id:
        return None

    try:
        payload = {
            "stock": f"{code},{org_id}",
            "category": "category_ndbg_szsh",    # 年度报告
            "pageNum": 1,
            "pageSize": 10,
            "tabName": "fulltext",
            "beginTime": f"{year}-01-01",
            "endTime": f"{year+1}-04-30",        # 年报披露截止4月底
            "isHLtitle": True,
        }
        resp = requests.post(
            self.CNINFO_SEARCH_URL,
            data=payload,
            headers=self.HEADERS,
            timeout=15,
        )
        items = resp.json().get("announcements") or []
        # 过滤出"年度报告"（排除摘要、社会责任报告等）
        items = [
            i for i in items
            if "年度报告" in i.get("announcementTitle", "")
            and "摘要" not in i.get("announcementTitle", "")
            and "社会责任" not in i.get("announcementTitle", "")
        ]
        if not items:
            return None

        file_url = self.CNINFO_FILE_URL + items[0]["adjunctUrl"]
        return self._fetch_pdf(file_url, save_path)

    except Exception as e:
        logger.warning(f"巨潮下载异常: {e}")
        return None

```

```

def _get_cninfo_org_id(self, code: str, exchange: str) -> Optional[str]:
    """查询巨潮机构ID"""
    try:
        resp = requests.get(
            "http://www.cninfo.com.cn/new/data/query.json",
            params={"keyWord": code, "maxNum": 5},
            headers=self.HEADERS,
            timeout=10,
        )
        for item in resp.json().get("stocks", []):
            if item.get("code") == code:
                return item.get("orgId")
    except Exception as e:
        logger.warning(f"获取orgId失败: {e}")
    return None

def _download_from_tushare(
    self, ts_code: str, year: int, save_path: Path
) -> Optional[Path]:
    """Fallback: 从Tushare接口获取披露URL"""
    try:
        df = self.pro.disclosure_date(
            ts_code=ts_code,
            end_date=f"{year+1}0430",
            start_date=f"{year}0101",
        )
        if df is None or df.empty:
            return None
        # 只取年报行
        df = df[df["report_type"] == "年报"]
        if df.empty:
            return None
        url = df.iloc[0].get("url") or df.iloc[0].get("report_url")
        if not url:
            return None
        return self._fetch_pdf(url, save_path)
    except Exception as e:
        logger.warning(f"Tushare下载异常: {e}")
        return None

def _fetch_pdf(self, url: str, save_path: Path) -> Optional[Path]:
    try:
        resp = requests.get(url, headers=self.HEADERS, timeout=60, stream=True)
        resp.raise_for_status()
        with open(save_path, "wb") as f:
            for chunk in resp.iter_content(chunk_size=8192):

```

```
        f.write(chunk)
    return save_path
except Exception as e:
    logger.warning(f"PDF下载失败 {url}: {e}")
    return None
```